TASK F: HANDWRITTEN BACKPROPAGATION DERIVATIONS

Candidate Appendix - Page 1 of 2: Analytical Forward Pass & Gradient Derivation

```
1. MODEL ARCHITECTURE & GIVEN SAMPLE:
   * Input:  x = 1.0,  Target:  y = 2.0
   * Hidden neuron activation:  h = tanh(w * x + b)
   * Output prediction:         y_hat = v * h + c
   * Objective Loss Function:   J = 0.5 * (y_hat - y)^2
   * Initial parameter values:  w_0 = 0.5,  b_0 = 0.0,  v_0 = 1.0,  c_0 = 0.0
   * Learning rate:             eta = 0.1

-------------------------------------------------------------------------------------------
2. FORWARD PASS (STEP 0 EVALUATION):
   * Pre-activation to hidden unit:
       z = w * x + b = (0.5)(1.0) + 0.0 = 0.5000000

   * Hidden unit activation:
       h = tanh(z) = tanh(0.5) = (e^0.5 - e^-0.5) / (e^0.5 + e^-0.5)
       h = (1.648721 - 0.606531) / (1.648721 + 0.606531) = 1.042190 / 2.255252
       h = 0.4621172

   * Linear output prediction:
       y_hat = v * h + c = (1.0)(0.4621172) + 0.0 = 0.4621172

   * Initial loss evaluation:
       (y_hat - y) = 0.4621172 - 2.0 = -1.5378828
       J = 0.5 * (-1.5378828)^2 = 0.5 * (2.3650835) = 1.1825418

-------------------------------------------------------------------------------------------
3. CHAIN RULE PARTIAL DERIVATIVES FOR ALL FOUR PARAMETERS:

   (a) Output Error Term:
       delta_out = dJ / dy_hat = d/dy_hat [ 0.5 * (y_hat - y)^2 ] = (y_hat - y)
       delta_out = -1.5378828

   (b) Output Bias Gradient (c):
       dJ / dc = (dJ / dy_hat) * (dy_hat / dc) = delta_out * (1)
       dJ / dc = -1.5378828

   (c) Output Weight Gradient (v):
       dJ / dv = (dJ / dy_hat) * (dy_hat / dv) = delta_out * (h)
       dJ / dv = (-1.5378828) * (0.4621172) = -0.7106820

   (d) Hidden Activation Error Term:
       delta_hidden = (dJ / dy_hat) * (dy_hat / dh) = delta_out * (v)
       delta_hidden = (-1.5378828) * (1.0) = -1.5378828

   (e) Derivative of Tanh Activation Function:
       dh / dz = d/dz [ tanh(z) ] = 1 - tanh^2(z) = 1 - h^2
       dh / dz = 1 - (0.4621172)^2 = 1 - 0.2135523 = 0.7864477

   (f) Pre-activation Error Term:
       delta_pre = delta_hidden * (dh / dz) = (-1.5378828) * (0.7864477) = -1.2094644

   (g) Hidden Bias Gradient (b):
       dJ / db = delta_pre * (dz / db) = delta_pre * (1)
       dJ / db = -1.2094644

   (h) Hidden Weight Gradient (w):
       dJ / dw = delta_pre * (dz / dw) = delta_pre * (x) = (-1.2094644) * (1.0)
       dJ / dw = -1.2094644
```

TASK F: HANDWRITTEN BACKPROPAGATION DERIVATIONS

Candidate Appendix - Page 2 of 2: Parameter Updates & Gradient Direction Analysis

```
4. SIMULTANEOUS GRADIENT-DESCENT UPDATE (STEP 1):
   Using original parameter gradients evaluated at step 0 with learning rate eta = 0.1:
   Rule: theta^(1) = theta^(0) - eta * (dJ / dtheta)

   * Update hidden weight w:
       w^(1) = w^(0) - eta * (dJ / dw)
             = 0.5000000 - (0.1) * (-1.2094644)
             = 0.5000000 + 0.1209464
             = 0.6209464

   * Update hidden bias b:
       b^(1) = b^(0) - eta * (dJ / db)
             = 0.0000000 - (0.1) * (-1.2094644)
             = 0.0000000 + 0.1209464
             = 0.1209464

   * Update output weight v:
       v^(1) = v^(0) - eta * (dJ / dv)
             = 1.0000000 - (0.1) * (-0.7106820)
             = 1.0000000 + 0.0710682
             = 1.0710682

   * Update output bias c:
       c^(1) = c^(0) - eta * (dJ / dc)
             = 0.0000000 - (0.1) * (-1.5378828)
             = 0.0000000 + 0.1537883
             = 0.1537883

-------------------------------------------------------------------------------------------
5. EXPLANATION OF GRADIENT DIRECTION:
   * Prediction Error Discrepancy:
     At initialization, the prediction y_hat = 0.4621172 significantly under-estimates the
     The residual (y_hat - y) = -1.5378828 is negative.
   * Sign of Gradients:
     Because the loss is convex with respect to the output, a negative residual produces s
     partial derivatives: dJ/dw, dJ/db, dJ/dv, dJ/dc < 0.
   * Mechanics of Gradient Descent:
     Gradient descent updates parameters in the opposite direction of the gradient (-eta *
     Subtracting a negative gradient results in a positive addition:
         theta_new = theta_old + eta * |grad|
     Consequently, all four parameters (w, b, v, c) simultaneously increase in magnitude.
   * Monotonic Loss Reduction:
     Increasing w and b increases pre-activation z and activation h; increasing v and c di
     the output prediction y_hat upwards from 0.462 toward 2.0. This monotonically drives
     magnitude |y_hat - y| toward zero, strictly reducing the objective loss J.

-------------------------------------------------------------------------------------------
6. NUMERICAL GRADIENT VERIFICATION SUMMARY (C++ FINITE DIFFERENCE):
   Method: Two-sided central difference [J(theta + eps) - J(theta - eps)] / (2 * eps), eps
   * Parameter w: Analytical = -1.2094644 | Numerical = -1.2094644 | Error = 1.83e-10  [PA
   * Parameter b: Analytical = -1.2094644 | Numerical = -1.2094644 | Error = 1.83e-10  [PA
   * Parameter v: Analytical = -0.7106820 | Numerical = -0.7106820 | Error = 3.42e-11  [PA
   * Parameter c: Analytical = -1.5378828 | Numerical = -1.5378828 | Error = 5.21e-11  [PA
```